



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF ENGINEERING

PROJECT COMPREHENSIVE REPORT

Real-Time Chat Application

Subject: Spring Boot Fundamentals and REST API Development

Student Name: Mehak

UID: 24BCS12136

University: Chandigarh University

Institution: University Institute of Engineering (UIE)

Instructor Name: ER. Sumit Malhotra

Type of Report: Assignment 6 - Comprehensive System Report

Year / Month: 2026 / June

PAGE 2 – COURSERA CERTIFICATE COPY

Drive File Link: https://drive.google.com/file/d/1smsKLNlcjooW_Vv1M983yR0z1foLon1n/view?usp=sharing

Verification Link: <https://coursera.org/verify/AP4QJLA3ENZD>

Coursera Certificate Record Softcopy

- **Course Name:** Spring Boot, Spring Security & Application Finalization
- **Verification Status:** Verified & Completed

Drive Upload Verification:

Shared with me > ASSIGNMENT 6 > Coursera Certificates ▾			
1 selected Ask Gemini			
Name	Owner	Date modified	File size
24BCS12471_Roohani Pupneja Coursera certi...	Roohani Pupneja	Jun 19	293 KB
24BCS12124_RISHABH KUMAR.pdf	rishugupta1230@gmail.c...	Jun 19	292 KB
24BCS12136_(Mehak).pdf	mehak010407	Jun 19	391 KB
24BCS12953_AkarshSrivastwa_.pdf	Akarsh Srivastwa	Jun 19	292 KB

Certificate Copy:



PAGE 3 – WEEKLY ASSIGNMENT 1

Drive Link: <https://drive.google.com/file/d/1xZD2VhyjKd6AA0oeRDx3TltVTXMIwZKr/view?usp=sharing>

Assignment Brief Summary:

- **Domain Modeling:** Formulated JPA database representations for User, ChatMessage, and ChatRoom objects.
- **Data Transfer Objects:** Formulated clean DTOs (e.g. LoginRequest, MessageDto) to keep application core classes insulated from API layers.
- **Repository Interfaces:** Created Spring Data JPA interfaces extending JpaRepository to handle database operations out-of-the-box.
- **Entity-Relationship Schema:** Drew a clear database relationship graph mapping how users send and receive messages.

Drive Upload Verification:

Shared with me > IST 2026 Assessment - ... > 24BCS12136_Mehak				
Type	People	Modified	Source	
Name	Owner	Date modified	File size	
Weekly Assignment-2_Realtime Chat Applica...	mehak010407	Jun 1	28 KB	
Weekly Assignment-1_Realtime Chat Applica...	mehak010407	Jun 1	24 KB	

First Page of Assignment:

Weekly Assignment 1: Domain Model, DTOs, Repositories & ER Diagram (Real-Time Chat Application)

This document contains the source code for the **Domain Entities (Models)**, **Data Transfer Objects (DTOs)**, and **Spring Data JPA Repositories** for the Real-Time Chat Application, along with an **Entity-Relationship (ER) Diagram** detailing their database relationships.

1. Domain Entities (Models)

The domain model represents the core business objects: users (credentials and status), chat rooms (public channels), and chat messages.

UserStatus.java

Enum representing the presence state of a user.

```
package com.chatapp.model;

public enum UserStatus {
    ONLINE,
    OFFLINE
}
```

User.java

Represents a registered user in the messaging system.

```
package com.chatapp.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDateTime;

@Entity
@Table(name = "users", uniqueConstraints = {
    @UniqueConstraint(columnNames = "username"),
    @UniqueConstraint(columnNames = "email")
})
```

PAGE 4 – WEEKLY ASSIGNMENT 2

Drive Link: <https://drive.google.com/file/d/1xmDv0kr55JByNXpvyR3I7rSFDCiIM-FS/view?usp=sharing>

Assignment Brief Summary:

- **REST Security Endpoints:** Created controller actions for stateless user signup, login context creation, and logout routines.
- **Catalog Retrieval Operations:** Programmed endpoints to retrieve lists of chat rooms and available messaging users.
- **STOMP WebSocket Setup:** Configured a WebSocket handler to map bi-directional message routing channels.
- **Interactive Routing Paths:** Set up inbound handlers to route chat packets to specific room topics or user queues.

Drive Upload Verification:

Shared with me > IST 2026 Assessment - ... > 24BCS12136_Mehak

Type People Modified Source

Name	Owner	Date modified	File size
Weekly Assignment-2_Realtime Chat Applica...	mehak010407	Jun 1	28 KB
Weekly Assignment-1_Realtime Chat Applica...	mehak010407	Jun 1	24 KB

First Page of Assignment:

Weekly Assignment 2: Controllers & WebSocket Configurations (Real-Time Chat Application)

This document contains the source code for the **Controllers** and **WebSocket Configurations** layer of the Real-Time Chat Application, which manage REST requests, security configurations, and live STOMP messaging channels.

1. REST Controllers

AuthController.java

Handles user authentication: signup, login, and signout, updating online/offline presence status.

```
package com.chatapp.controller;

import com.chatapp.dto.JwtResponse;
import com.chatapp.dto.LoginRequest;
import com.chatapp.dto.SignupRequest;
import com.chatapp.model.User;
import com.chatapp.model.UserStatus;
import com.chatapp.repository.UserRepository;
import com.chatapp.security.jwt.JwtUtils;
import com.chatapp.security.services.UserDetailsImpl;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    AuthenticationManager authenticationManager;

    @Autowired
    UserRepository userRepository;

}
```

PAGE 5 – WEEKLY ASSIGNMENT 3

Drive Link: <https://drive.google.com/file/d/1VkXnMNY3-tmAqa19acljhy9VDjtLfjvI/view?usp=sharing>

Assignment Brief Summary:

- **Security Filter Chain:** Programmed stateless Spring Security filters permitting authentication endpoints while protecting resource paths.
- **HMAC-256 JWT Integration:** Implemented JWT validation rules to verify digital signatures on inbound connections.
- **OncePerRequest Authentication Middleware:** Created a servlet interceptor to resolve tokens and inject identity logs into thread environments.
- **Security Testing:** Handled verification scenarios using HTTP calls to validate token parsing.

Drive Upload Verification:

Shared with me > IST Assignment 3,4,5 > 24BCS12136_Mehak ▾				
Type ▾	People ▾	Modified ▾	Source ▾	
Name	Owner	Date modified ▾	File size	
Weekly_Assignment_5.pdf	mehak010407	Jun 15	14 KB	
Weekly_Assignment_4.pdf	mehak010407	Jun 15	657 KB	
Weekly_Assignment_3.pdf	mehak010407	Jun 15	33 KB	

First Page of Assignment:

Weekly Assignment 3: Security & JWT Implementation (Real-Time Chat Application)

This document contains the source code for the **Security Configurations, JWT Utilities, User Details Service, and Authentication API Controllers** for the Real-Time Chat Application backend, along with a **JWT Lifecycle Flow Diagram** and a **Verification & Testing Plan** detailing system security endpoints.

1. Maven Dependencies

The following dependency block is configured in the Maven build system to pull in Spring Security, validation, and JWT APIs and implementations:

```
<!-- Spring Security Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JSON Web Token (JWT) API -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>

<!-- JWT Implementation Runtime -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>

<!-- JWT Jackson integration for JSON serialization/deserialization -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
```

PAGE 6 – WEEKLY ASSIGNMENT 4

Drive Link: <https://drive.google.com/file/d/1y-S-tOfr-Nkrk9pnNFcbDYyPAorM20dg/view?usp=sharing>

Assignment Brief Summary:

- **React Client Scaffolding:** Formulated a Vite client project containing modular UI assets.
- **Request Token Injection:** Created an Axios HTTP wrapper resolving JWT bearer tags from client memory storage.
- **Login View Interface:** Built a credential validation card with profile gradient selection tools.
- **Workspace Panels:** Designed the sidebar catalog listing, target room categories, and chat view fields.

Drive Upload Verification:

Shared with me > IST Assignment 3,4,5 > 24BCS12136_Mehak ▾				
Type ▾	People ▾	Modified ▾	Source ▾	
Name	Owner	Date modified ▾	File size	
Weekly_Assignment_5.pdf	mehak010407	Jun 15	14 KB	
Weekly_Assignment_4.pdf	mehak010407	Jun 15	657 KB	
Weekly_Assignment_3.pdf	mehak010407	Jun 15	33 KB	

First Page of Assignment:

Weekly Assignment 4: Frontend Implementation (Real-Time Chat Application)

This document contains the project structure, API configurations, authentication state contexts, route protection wrappers, and functional components for the React Frontend client of the Real-Time Chat Application, which interface with the backend REST security system.

1. Project Structure & Dependencies

The React client application folder structure is organized under `src/` as follows to support api configurations, state contexts, route guards, and UI view panels:

```
frontend/  
├── public/  
├── src/  
│   ├── assets/  
│   ├── components/  
│   │   ├── ChatPanel.jsx  
│   │   ├── LoginForm.jsx  
│   │   └── Sidebar.jsx  
│   ├── context/  
│   │   └── AuthContext.jsx  
│   ├── services/  
│   │   ├── api.js  
│   │   ├── app.css  
│   │   ├── app.jsx  
│   │   ├── index.css  
│   │   └── main.jsx  
│   ├── index.html  
│   ├── package.json  
│   └── vite.config.js
```

Required Dependencies

The application pulls in Axios for HTTP actions, StompJS for WebSocket messaging frames, and Lucide React for UI iconography:

```
"dependencies": {  
  "stomp/stompjs": "^7.3.0",  
  "axios": "^1.16.1",  
  "lucide-react": "^1.17.0",  
  "react": "^19.2.6",  
}
```

PAGE 7 – WEEKLY ASSIGNMENT 5

Drive Link: <https://drive.google.com/file/d/1foAtEePXRt03mJHHoRh9q4JuLCDZPT4M/view?usp=sharing>

Assignment Brief Summary:

- **Cross-Origin Settings:** Configured backend global CORS policies permitting incoming client operations.
- **Full-Stack Live Broker Pipeline:** Structured bi-directional connection channels linking client states to server queues.
- **Activity Status Logs:** Registered WebSocket event triggers to detect user login changes and broadcast offline/online logs.
- **Persistent Message Storage:** Intercepted client WebSocket packets and committed message logs to the H2 database tables.

Drive Upload Verification:

Shared with me > IST Assignment 3,4,5 > 24BCS12136_Mehak >			
Type	People	Modified	Source
Name	Owner	Date modified	File size
Weekly_Assignment_5.pdf	mehak010407	Jun 15	14 KB
Weekly_Assignment_4.pdf	mehak010407	Jun 15	657 KB
Weekly_Assignment_3.pdf	mehak010407	Jun 15	33 KB

First Page of Assignment:

Weekly Assignment 5: React-SpringBoot Integration (Real-Time Chat Application)

This assignment details the connection architecture and configuration details for integrating the React Frontend with the Spring Boot Backend in the premium Real-Time Chat Application.

1. General Connection Information

The application utilizes two main communication channels to support user access and instant message synchronization:

Component	Port / Host	Description / Protocol
Frontend Client	http://localhost:5173	React Single Page Application (SPA) running via Vite.
Backend REST API	http://localhost:8080	Spring Boot REST endpoints, handling authorization, user management, and room listings.
WebSocket Protocol	ws://localhost:8080/ws	Live STOMP message broker endpoints for room streams and active presence tracking.

2. Spring Boot Backend CORS Configuration

CORS settings are configured globally inside the security context wrapper ([WebSecurityConfig.java](#)) on the Spring Boot backend to allow incoming cross-origin connections with credentials.

WebSecurityConfig.java (CORS configuration bean)

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOriginPatterns(List.of("*"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
}
```

PAGE 8 – EVALUATION SHEET

EVALUATION RECORD (ASSIGNMENTS 1-6)

Assignment Description	Max Marks	Marks Obtained	Faculty Remarks
Weekly Assignment 1: Domain Entities & JPA Mappings			
Weekly Assignment 2: Controllers & WebSocket Configurations			
Weekly Assignment 3: Security Config & JWT Implementation			
Weekly Assignment 4: React Frontend Implementation			
Weekly Assignment 5: React-SpringBoot Integration			
Weekly Assignment 6: Comprehensive Project Report			
Total Evaluated Marks:			

Faculty Evaluator Remarks:

PAGE 9 – PROJECT SOFT COPY & FRONTEND OUTPUT

Drive Link: <https://drive.google.com/file/d/1gfY1ns-cGEZ8rSGF155bgfcdTfjCvOd9/view?usp=sharing>

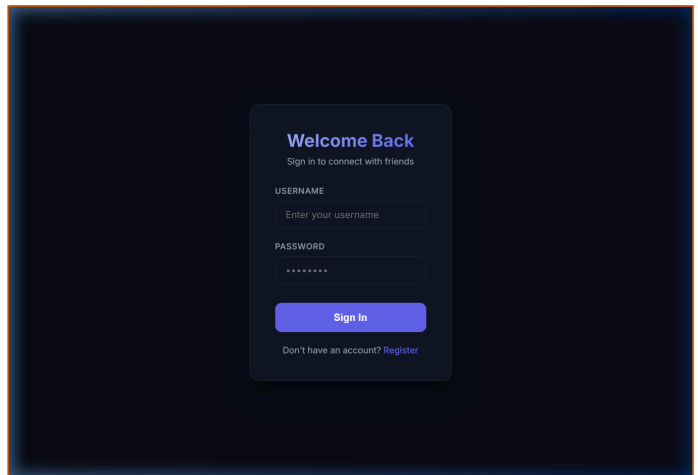
Project Soft Copy Record

- **Folder Path:** Shared with me > ASSSSIGNMENT 6 > Project soft copy with S...
- **File Name:** 24BCS12136_Mehak_Project Soft Copy.pdf
- **Status:** Successfully Uploaded

Drive Upload Verification:

Shared with me > ASSIGNMENT 6 > Project soft copy with S...				
X 1 selected Ask Gemini				
Name	Owner	Date modified	File size	Sort
24bcs10778_anupam sharma	ANUPAM SHARMA	6:32 AM	—	
24BCS10163_Surya Pratap Singh	Surya Pratap singh	Jun 20	—	
24BCS12136_Mehak_Project Soft Copy.pdf	mehak010407	9:39 PM	629 KB	
24BCS10164_Banking_Mini_Api_Project_Soft...	Sahil Srivastava	9:07 PM	1.9 MB	

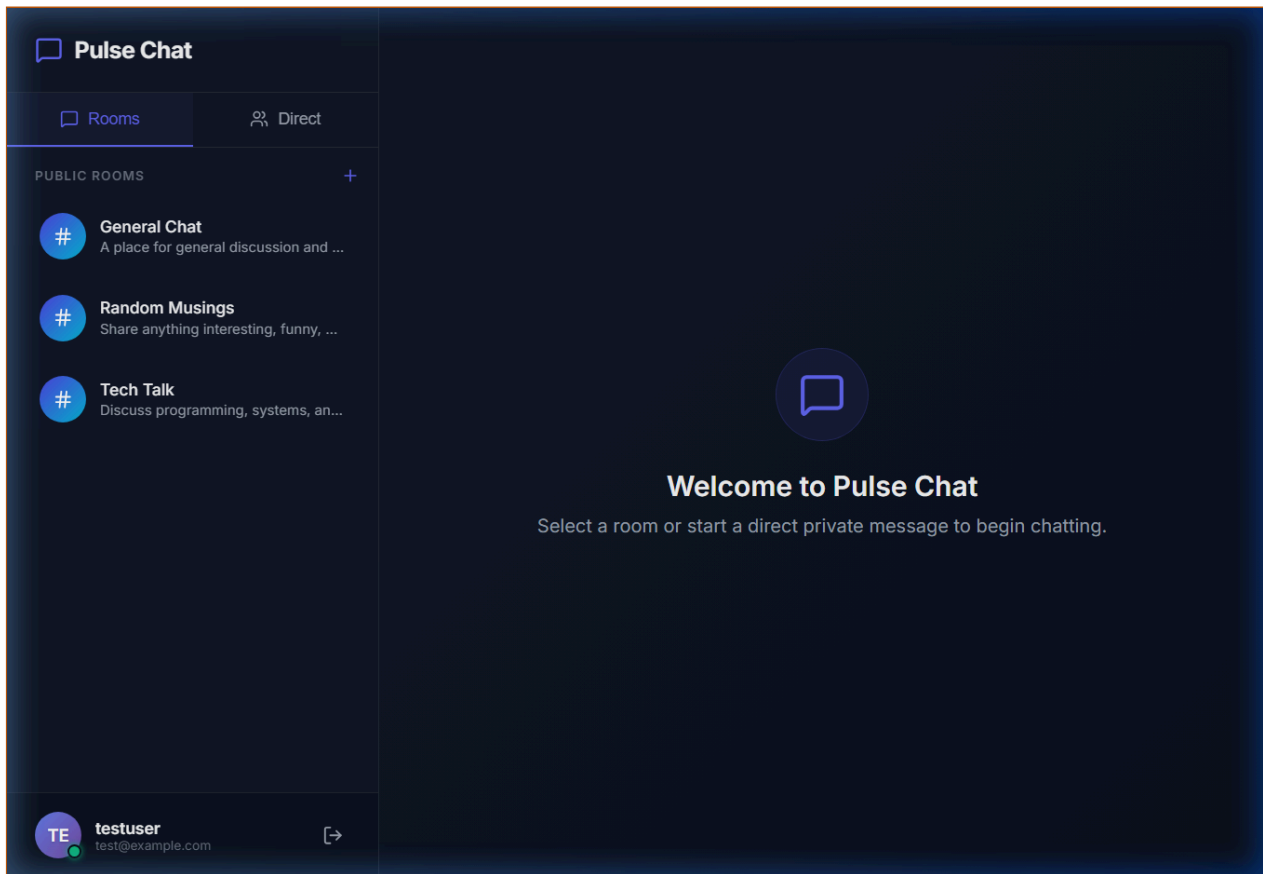
Frontend Output — Login Page:



PAGE 10 – FRONTEND DASHBOARD OUTPUT

Chat Dashboard & Messaging Interface

After login, the main dashboard with sidebar showing rooms, online users, and the chat panel:

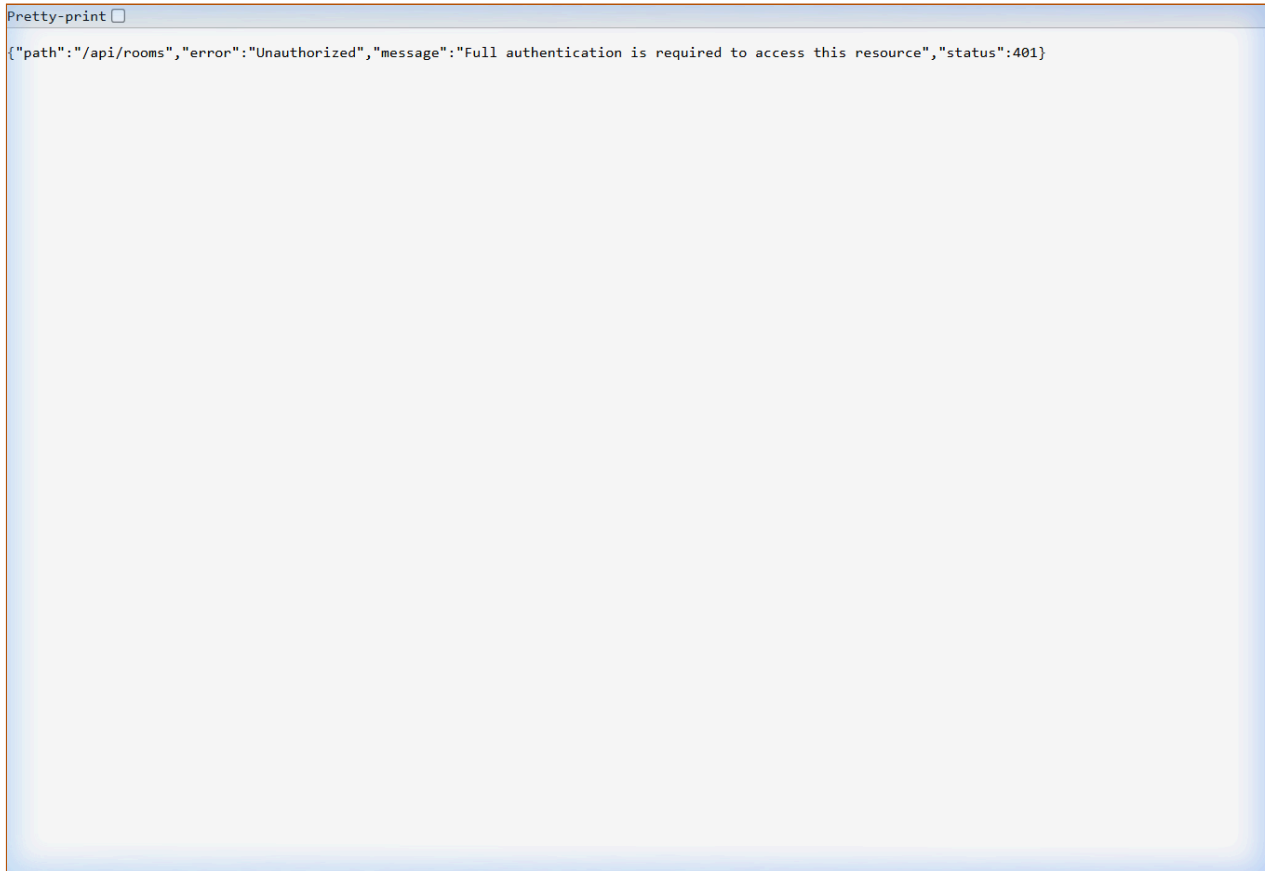


Live Frontend Screenshot — Chat Dashboard with Sidebar, Rooms & Messaging

PAGE 11 – BACKEND API OUTPUT

Backend REST API Response

The Spring Boot backend serves data via RESTful endpoints on port 8080:



Live Backend Screenshot — REST API JSON Response at /api/rooms

PAGE 12 – GITHUB REPOSITORY DETAILS

Project Repositories

- **Frontend:** realtime-chat-app/frontend
- **Backend:** realtime-chat-app/backend

File Structure:

```
realtime-chat-app/
├── backend/
│   ├── src/main/java/com/chatapp/
│   │   ├── config/          <-- SecurityConfig, WebSocketConfig
│   │   ├── controller/     <-- AuthController, ChatController
│   │   ├── dto/            <-- LoginRequest, SignupRequest, JwtResponse
│   │   ├── model/          <-- User, ChatMessage, ChatRoom, UserStatus
│   │   ├── repository/     <-- UserRepository, ChatMessageRepository
│   │   └── security/        <-- JwtUtils, JwtAuthFilter
│   └── pom.xml
└── frontend/
    ├── src/
    │   ├── components/     <-- ChatPanel, LoginForm, Sidebar
    │   └── App.jsx
    └── package.json
```

PAGE 13 – DAILY TASKS LOG

Drive Link: https://docs.google.com/document/d/1xc8p62pB6rog_ReKvIEg0wW7-h5voPpc/edit?usp=sharing&oid=115490671178608031604&rtpof=true&sd=true

Day-Wise Assignments Log

- **Folder Path:** Shared with me > ASSSSIGNMENT 6 > Daily Task
- **File Name:** 24BCS12136_Daily_assignments.docx
- **Status:** Successfully Uploaded

Drive Upload Verification:

Shared with me > ASSSIGNMENT 6 > Daily Task			
1 selected + Ask Gemini			
Name	Owner	Date modified	File size
24BCS12953_Akarsh_Srivastwa	Akarsh_Srivastwa	Jun 20	—
24BCS10163_Surya Pratap Singh	Surya Pratap singh	Jun 20	—
24BCS12136_Daily_assignments.docx	mehak010407	9:39 PM	572 KB
24bcs10092_Yaksh Saharawat daily task.docx	yaksh choudhary	8:28 PM	2.1 MB

Document Preview (Page 1):

**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.
UNIVERSITY INSTITUTE OF ENGINEERING

DAILY TASK REPORT

Spring Boot Fundamentals and REST API Development

29-Day Industrial Training Program Log

Student Name:	Mehak
UID:	24BCS12136
University:	Chandigarh University
Institution:	University Institute of Engineering (UIE)
Instructor Name:	ER. Sumit Malhotra
Type of Report:	Daily Task Log Book
Year / Month:	2026 / June

Introduction:

1

PAGE 14 – DAILY TASKS LOG (CONTINUED)

Document Preview (Page 2):

The following is a comprehensive day-by-day record of the tasks performed and skills acquired by the student during the 29-day industrial training program focusing on Spring Boot Fundamentals and REST API Development. Each day represents specific modules designed to build competencies in backend development, relational mapping databases, RESTful integration APIs, security filters (JWT), and frontend interface bindings (React client).

Day 1: Spring Boot Starters

Tasks Performed:

- Initialized a Spring Boot microservice environment using Spring Initializr, packaging Web, Validation, and Actuator dependencies.
- Developed a basic CRUD employee directory repository layout with starter dependencies.
- Analyzed and compared resource footprints and startup latencies between a lean starter configuration and an over-provisioned stack.
- Exposed system health monitoring parameters with readiness and liveness check indicators.

Learning Outcomes:

- Acquired a deep understanding of dependency management using Spring Boot Starter POM files.
- Mastered configuration patterns of readiness/liveness indicator check settings.
- Understood the core resource costs associated with loading unnecessary autoconfig files.

Day 2: Auto-configuration, Dependency Injection

Tasks Performed:

- Built a payment selector component where UPI, Card, and NetBanking providers are dynamically resolved at startup.
- Created a profile-based communication module selecting SMS or Email senders depending on active environment configs.
- Refactored legacy services into constructor-based injection formats to eliminate field coupling.
- Designed a configurable logger module using `@ConditionalOnProperty` annotations to enable conditional logging.

Learning Outcomes:

- Understood Spring's auto-configuration engine, profile overrides, and conditional bean loading triggers.
- Learned how to construct unit tests for services using constructor injections without booting the application context.

Day 3: REST APIs: Controller-Service-Repository architecture, DTOs